

Project ATLAS

Explained Simply — What It Is, Why It Matters, and the Math Behind It

A plain-language walkthrough of the problem ATLAS solves, how it solves it, the actual mathematics underneath each piece with real-world comparisons, how it would actually get built, and an honest look at what could go wrong.

*Draft explainer document — not the investment memo or the technical spec.
Prepared July 2026.*

Table of Contents

1. Why Do We Need This?
2. What Exactly Is Broken? (The Problem, Precisely)
3. The Big Idea: Data Has a Shape
4. Comparing Notes Without Sharing Secrets: The TIP Handshake
5. Computing Without Copying: S-MCP
6. A Worked Example, With Real Numbers
7. How This Actually Gets Built
8. The Honest Take: Where This Could Fail

1. Why Do We Even Need This?

Today's AI systems answer questions by turning documents into long lists of numbers (embeddings) and finding the list that looks most similar to your question. This works fine for simple lookups, but it breaks down in three specific ways:

- It can't follow a chain of clues across several documents — it only compares one thing to one other thing at a time.
- It can't notice when something is structurally unusual (like fraud), because unusualness is about how a thing relates to its neighbors, not just its raw distance from a query.
- It has no safe way for two separate companies (say, two banks) to reason over each other's data without physically sending the data over — which is illegal or forbidden in regulated industries.

IN EVERYDAY TERMS

Think of two detectives working the same case from two different cities, each holding half the case file. Neither is allowed to fax the other their evidence, but they still need to know if their leads point to the same suspect. Right now there's no good way to check that without breaking the rules. ATLAS is a way to compare the “shape” of each detective's evidence without either one showing the other their file.

ATLAS's answer is to stop treating data as flat text and start treating it as a geometric shape — a manifold — that can be compared, traversed, and reasoned over the way you'd reason over a map, not a word search.

2. What Exactly Is Broken? (The Problem, Precisely)

Standard retrieval systems (Retrieval-Augmented Generation, or RAG) embed each document as a vector, then rank documents by cosine similarity to the query vector.

THE MATH, MADE CONCRETE — Cosine Similarity

$$\cos(\theta) = (A \cdot B) / (||A|| * ||B||)$$

where A = query vector, B = document vector,

$A \cdot B$ = dot product, $||A||$ = vector length

Real-world comparison: this is like judging how close two people are by measuring the angle between them and you, from where you're standing. It works for a single pair, but it never notices that three other people are actually standing in a circle holding hands two rooms over. That's the structural ceiling — no amount of extra data or a better embedding model changes what this one formula can express.

Three concrete failure modes follow directly from this:

- **Multi-hop questions** — an answer that requires chaining facts across records has no natural representation in a purely pointwise scoring system.
- **Structural anomalies** — fraud and defects are often unusual in their relationship to their neighborhood, not in raw distance from a query.
- **Cross-organization reasoning** — there is no mechanism for two organizations to jointly reason over data without centralizing it, which regulated industries cannot do.

3. The Big Idea: Data Has a Shape

ATLAS's first pillar, Manifold-Core, builds an actual geometric structure out of a dataset instead of a flat list of vectors. It does this in five steps:

- 1 Encode records into embeddings (same starting point as ordinary RAG).
- 2 Build an approximate k-nearest-neighbor graph connecting each point to its closest neighbors.
- 3 Grow a Vietoris–Rips filtration — connect points as an allowed distance is slowly increased.
- 4 Track which shapes (clusters, loops, voids) survive as that distance grows, versus which vanish quickly.
- 5 Answer a query by dropping it into this shape and letting it settle, like a marble rolling into a groove.

IN EVERYDAY TERMS

Picture a subway map instead of a straight-line ruler. Two stations can be geographically close but useless to compare directly if there's no track between them, while two stations far apart might be one quick transfer away because of how the lines loop and connect. ATLAS builds that map first, then asks "which route does this question travel," instead of just measuring straight-line distance.

THE MATH, MADE CONCRETE — Vietoris–Rips Complex and Persistence

$$VR_e(X) = \{ \text{sigma (subset of) } X : d(x_i, x_j) \leq e \text{ for all } x_i, x_j \text{ in sigma} \}$$

As e (epsilon) increases from 0 to infinity, simplices are added one by one.

Each topological feature (a component, loop, or void) is 'born' at some threshold b and 'dies' (gets filled in / merges) at some threshold d .
 $\text{persistence}(\text{feature}) = d - b$

Real-world comparison: imagine slowly filling a room with fog. Small gaps between furniture disappear into the fog almost immediately — that's noise. But a real hallway stays visibly open for a long time before the fog finally swallows it — that's a persistent, structurally real feature. The number $(d - b)$ is just "how long did this hallway stay open before the fog got it," and that single number is what separates real structure from noise, mathematically, instead of by a hand-tuned guess.

Retrieval then isn't 'find the nearest point.' A query is inserted into the complex, and a potential field is defined using local point density plus nearby persistence weights. The query traverses that field — conceptually similar to gradient descent — so its answer can pass through several related regions in sequence. That is the mechanism that gives multi-hop reasoning a native representation.

WHERE THIS GETS HARD

Exact Vietoris–Rips persistence computation scales combinatorially in the number of simplices as dataset size and dimension grow. This is the single biggest open engineering problem in the whole system. Mitigations under investigation — sparse/witness-complex approximations, capping homology at low dimensions, and incremental updates — are reasonable, literature-backed ideas. None of them has yet been proven sufficient at production scale; that is exactly what the first implementation phase is designed to measure.

4. Comparing Notes Without Sharing Secrets: The TIP Handshake

TIP (Topology Interchange Protocol) lets two organizations discover that they have matching structure in their data — without either one sending the other any raw records. It works on a *local* neighborhood around one specific query, not the whole dataset (an earlier, looser description of this protocol compared whole-dataset summaries; that version doesn't work, because a global summary can't say *where* an overlap is, only *that* the two datasets are shaped similarly overall).

IN EVERYDAY TERMS

It's like two witnesses independently describing a getaway car to police, in separate rooms, without comparing notes first. If both descriptions match closely enough — same color, same dent, same direction — the police know they're looking at the same car, without either witness ever seeing the other's statement.

The four real steps:

- 1 Restrict the topology computation to the k-nearest neighborhood of the query point only.
- 2 Convert that local persistence diagram into a fixed-length vector using a persistence landscape transform.
- 3 Hash that vector using locality-sensitive hashing (random hyperplane projection).
- 4 Exchange only the hash codes. A collision resolves to a locally-scoped region id, meaningless outside the org that generated it.

THE MATH, MADE CONCRETE — Persistence Landscape + Locality-Sensitive Hashing

Persistence landscape (turns a diagram into a comparable vector):

$\lambda_k(t) = k\text{-th largest value of } \min(t - b_i, d_i - t) \text{ over all birth/death pairs } (b_i, d_i), \text{ for a grid of } t \text{ values}$

Random-hyperplane LSH (turns a vector into a short, one-way hash):

$h_r(v) = \text{sign}(v \cdot r)$, r drawn randomly from a unit sphere

$\Pr[h_r(v_1) = h_r(v_2)] = 1 - \theta(v_1, v_2) / \pi$

Real-world comparison: think of it like two people each cutting their own key from a private blueprint, then only comparing the silhouette of the cut, never the blueprint itself. $\theta(v_1, v_2)$ is the angle between the two vectors — the more alike two organizations' local shapes are, the smaller that angle, and the higher the chance their key silhouettes match. The original shape can't be reconstructed from the silhouette alone.

SAID PLAINLY

This is probabilistic matching, not a cryptographic proof. False positives (coincidental collisions) and false negatives (near-matches that don't collide) are expected outcomes of the design and must be measured, not assumed away. A collision also leaks a small amount of information — the fact that a structural match exists — so 'zero-knowledge' is not the right term for this; 'privacy-preserving approximate matching' is. Repeated querying over time has no proven leakage bound yet; rate-limiting plus differential-privacy noise in the hashing step is the planned mitigation, not a solved problem.

5. Computing Without Copying: S-MCP

Once TIP finds a matching region, someone still needs to actually compute an answer using the data sitting at that region — without that data ever leaving its home organization. Sovereign-MCP (S-MCP) solves this by sending the *computation* to the data, instead of the data to the computation.

IN EVERYDAY TERMS

It's the difference between mailing someone your house keys versus having a background-checked inspector visit, look only at the one room they're licensed to inspect, write a short report, and leave — with no key ever changing hands and no memory of anything else in the house.

The execution flow:

- 1 A small, task-distilled inference module (not a general model) is trained offline for one narrow sub-question.
- 2 It's compiled to WebAssembly (wasm32-wasi), quantized to int8, and signed.
- 3 The host verifies the signature and runs it in a sandbox with a fixed memory ceiling, a fixed compute-step budget, and no filesystem or network access — only a narrow, read-only view of the matched region.
- 4 It returns a small result vector (at most 256 bytes).
- 5 The sandbox is torn down completely; no state persists.

There isn't new math here so much as a discipline: the module has already been told exactly where to look and what narrow question to ask, by TIP, before it is ever invoked — so it needs accuracy on one distilled task, not general reasoning.

6. A Worked Example, With Real Numbers

Two banks each suspect a shell company. Bank A and Bank B each hold a cluster of wire transfers around a suspicious account. Here's a simplified run-through with toy numbers, following the same steps used in the real design.

Step 1 — Local persistence.

Bank A's cluster produces one persistent loop with birth $b = 0.10$ and death $d = 0.85$. Persistence $= 0.85 - 0.10 = 0.75$ — a strong, real feature (not noise).

Bank B's cluster, around its own suspicious account, produces a similarly-shaped loop with birth $b = 0.12$ and death $d = 0.9$, persistence $= 0.78$.

Step 2 — Landscape vectors.

Each bank converts its diagram into a landscape vector, say a 4-dimensional vector sampled at $t = 0.2, 0.4, 0.6, 0.8$:

$$v_A = (0.10, 0.30, 0.30, 0.10) \quad v_B = (0.08, 0.29, 0.31, 0.12)$$

Step 3 — Hashing and collision probability.

The angle between v_A and v_B is small (the vectors point in nearly the same direction), roughly $\theta = 8$ degrees $= 0.14$ radians. The single-hyperplane collision probability is:

$$\Pr[\text{collision}] = 1 - \theta/\pi = 1 - 0.14/3.1416 \approx 1 - 0.045 = 0.955$$

Using several independent hyperplanes (say 20) and requiring most of them to agree pushes the combined false-positive rate down further, at the cost of also slightly raising the chance of missing a genuine match — a tunable trade-off, not a free win.

Step 4 — Sandboxed execution.

The collision resolves to a region id at each bank. A signed, distilled entity-linkage Wasm module (say, capped at 8 MB memory and 50 million compute steps) runs against only that region at each bank, and each returns a short result vector, at most 256 bytes, that a regulator combines to identify the shared entity — having received zero raw transaction records from either bank.

7. How This Actually Gets Built

Phase 1 — Manifold-Core (Months 1–3)

- Corpus: 10k–50k documents, embedded with an off-the-shelf sentence-transformer — no custom encoder, since that isn't the bottleneck.
- Build an approximate k-NN graph with hnswlib or faiss-HNSW, benchmarked for recall@k against a brute-force baseline on a small subsample.
- Run Vietoris–Rips + persistent homology through GUDHI or Ripser — never hand-rolled; this is a mathematically subtle area with decades of prior work already solved.
- Validate against synthetic ground truth first (a circle → exactly one persistent loop; two clusters → exactly two persistent components) before touching real data.
- Benchmark traversal-retrieval against plain cosine-similarity RAG on named external multi-hop datasets (HotpotQA, MuSiQue) — not a self-invented comparison.

Phase 2 — S-MCP (Months 3–5)

- Distill a small inference module from a larger teacher model for one narrow task, quantize to int8, compile to wasm32-wasi, sign it.
- Run it in Wasmtime under WASI capabilities: fixed memory ceiling, fixed compute-step budget, no filesystem or network access.
- Prove zero data exfiltration independently — a public demo repository, not an internal claim.

Phase 3 — TIP (Months 5–8)

- Local VR complex around a query point → persistence landscape transform → random-hyperplane LSH → exchange only hash codes over QUIC (tokio + quinn).
- Deploy 3 simulated nodes, run the two-bank worked example end-to-end, then commission an independent security review of the sandbox boundary before any real pilot.

That's a legitimate, buildable sequence. Nothing in it requires inventing new math or new sandboxing technology — it's integration of existing, peer-reviewed tools.

8. The Honest Take: Where This Could Fail

The roadmap above is coherent, but here is where it's genuinely fragile — stated directly rather than softened.

1. THE CORE BOTTLENECK IS UNSOLVED, AND EVERYTHING SITS ON TOP OF IT

Exact Vietoris–Rips persistence is combinatorially expensive in simplex count. The planned mitigations — sparsification, low-dimension capping, incremental updates — are research questions, not scoped engineering tasks. You cannot confidently schedule 'Months 1–3' against a question whose answer you don't yet know. If sparsification doesn't hold up, the retrieval mechanism has no foundation, and Phases 2–3 become irrelevant.

2. THE TIMELINE ASSUMES THREE HARD RESEARCH DOMAINS GO SMOOTHLY, SEQUENTIALLY

A topologist, a Rust/QUIC engineer, and a Wasm/sandbox engineer, hired one after another, are expected to take unproven TDA-at-scale, a from-scratch privacy protocol, and a sandboxed distillation pipeline from zero to a live three-node demo in eight months. Each of those alone is a plausible research program on its own. Stacked with gated hiring, the timeline is optimistic — and the plan's own phase-gating quietly concedes this by treating funding as re-earned each phase, not committed up front.

3. 'ZERO-KNOWLEDGE' WAS ALREADY WALKED BACK ONCE, AND THE LEAK IS REAL

The corrected design admits a collision leaks an approximate similarity signal, and repeated querying has no proven bound on cumulative leakage yet. Rate-limiting plus differential-privacy noise is plausible, but unproven for this specific construction — and in the bank example, that's exactly the leakage a regulator or examiner would test first.

4. LSH IS PROBABILISTIC, AND THE MISSED-MATCH RATE MATTERS HERE SPECIFICALLY

A roughly 95% single-hyperplane collision probability sounds strong, but false negatives compound: a bank clears its own compliance bar by finding a match, yet still misses one it should have found — and this is fraud detection, where a missed match isn't a UX inconvenience, it's a shell company slipping through undetected.

5. THE 30% ACCURACY TARGET IS A PLACEHOLDER, NOT A PREDICTION

It's a reasonable planning stake, but it was picked before the experiment was run, not derived from one. Treat it, and communicate it to investors or a team, as a target to test against — not a number already earned.

None of this means the architecture is wrong. The willingness to state these as open problems, rather than solved ones, is itself the strongest signal that the risk is understood. The accurate summary: the roadmap is buildable, the timeline assumes several unresolved research questions

resolve favorably, and Phase 1's benchmark is the real go/no-go decision point — not a formality on the way to Phase 2.

This document is a plain-language and mathematical explainer draft based on the ATLAS technical README and roadmap materials. It is not the investment memorandum and makes no claims beyond what those source documents state.